

Passing Every Test While Completely Broken: How Single-Address-Space Test Doubles Structurally Hide Distributed Coordination Bugs

Abstract

1. Introduction
2. Background: the runtime under test
3. The campaign: four rounds, seven defects
4. The taxonomy: why the tests were blind
5. Mitigations and their measured effect
6. Related work and discussion
7. Threats to validity
8. Conclusion

Passing Every Test While Completely Broken: How Single-Address-Space Test Doubles Structurally Hide Distributed Coordination Bugs

Research companion to the Facetwork thesis ([thesis.md](#)). Empirical basis: the July 2026 distributed-catch verification campaign; all commits, test suites, and workflow-state evidence cited here are in the Facetwork repository and its fleet's execution records.

Abstract

We report on a language feature — the FFL `catch` clause, error recovery for distributed workflow steps — that passed 100% of its unit and end-to-end tests while having **never once worked** in distributed execution. Verifying it live took four fix-and-rerun rounds on a production fleet and surfaced seven independent defects, closed by five fixes. **Five** of the seven map to a specific *semantic gap* between the in-memory persistence fake used by the test suite and the production database — object aliasing, index semantics, commit visibility, name-versus-identity resolution, and cache locality — and are the paper's focus; the remaining two are control-flow path-coverage artifacts that distributed execution exposed by selecting different code paths, which we include because they co-travel with the store-semantics gaps. We argue the five store-semantics gaps are not accidents of a lazily written fake but *structural properties of any single-address-space test double*, present a taxonomy for auditing them, and evaluate two mitigations adopted mid-campaign — same-assertion dual-store parity suites, and cheap live verification runs as first-class bug finders — which together caught the next feature's coordination divergences before deployment. The broader claim: for systems whose correctness lives in cross-process coordination, a feature is not done when its tests pass; it is done when it has *run distributed*, and the economics of verifying that are far better than folklore suggests.

1. Introduction

Distributed workflow engines occupy an awkward testing position. Their unit of correctness — a step lifecycle spanning claim, execution, state transition, and continuation across independently crashing processes — is exactly the thing an in-process test cannot instantiate. Standard practice substitutes a *test double* for the coordination substrate: an in-memory implementation of the persistence interface, letting the state machine, evaluator, and handlers execute in one address space at test speed.

Facetwork follows this practice. Its `MemoryStore` implements the same `PersistenceAPI` protocol as the production `MongoStore`; roughly 1,200 runtime tests execute against it, including multi-step end-to-end workflow executions. The practice is not naive: the interface is explicit, the fake is complete, and the suite is disciplined about exercising real state machines rather than mocks of them.

It was therefore instructive to discover that the `catch` clause — error recovery, a feature with dozens of dedicated unit tests plus end-to-end coverage, shipped and documented — **had never successfully executed on the fleet**. Every distributed invocation either silently skipped recovery, reported the recovery itself as failed, executed the wrong code inside the recovery block, completed the step with empty results while recovery ran into the void, or parked forever awaiting a sub-block the database query could not return. Seven distinct defects across four rounds; zero test failures throughout.

This paper makes four contributions:

1. A **case narrative** (§3) of the four-round live verification campaign that surfaced and fixed the seven defects, with per-round costs — evidence that live verification is cheap relative to its yield.
2. A **taxonomy of semantic gaps** (§4) between single-address-space fakes and real coordination substrates, with each defect mapped to its gap class and an analysis of why the test suite was structurally blind to it.
3. Two **mitigations with results** (§5): dual-store parity suites that run identical assertions against the fake and the real store, and verification runs as a methodology; both were applied to the next feature (execution environments) and caught its divergences pre-ship.
4. A **discussion** (§6) of why these gaps generalize beyond this system, and where the standard remedies (integration tests, chaos engineering, deterministic simulation) do and do not address them.

2. Background: the runtime under test

Facetwork executes workflows compiled from FFL, a declarative flow language. Three properties matter for this paper:

- **Leaderless, queue-coordinated execution.** Every runner is an autonomous process; all coordination is atomic document operations on MongoDB (`find_one_and_update` claims). There is no scheduler process to test against — the "scheduler" is the union of all runners' poll loops.
- **Persisted step state machines.** Each workflow step advances through a state machine (`FacetInit` → `Scripts` → `EventTransmit` → `Blocks` → `Capture` → `Complete`); state lives in the database, and *any* runner may process any step's next transition by claiming its continuation task. A step's processing is therefore distributed across processes and time.
- **The `catch` clause** attaches error recovery to steps, facets, and workflows: on failure, execution enters `CATCH_BEGIN`, a recovery sub-block (`AND_CATCH`) is created and executed, and its `yield` supplies the step's returns. The caught error is exposed to recovery code as `$.error` / `$.error_type`.

The test double: `MemoryStore` keeps steps, tasks, and servers in Python dictionaries guarded by a claim lock. It is a *fake* in Fowler's sense — a working implementation with real behavior — not a stub.

3. The campaign: four rounds, seven defects

The verification target was a production workflow (a 25-region continental atlas) including a region expected to fail (Greenland: no population-tagged cities qualify), with a region-level catch expected to degrade the failure to a disclosed exclusion. Success criteria: workflow completes; failed region appears as an excluded row; all other regions unaffected. Each round: run on the live fleet (~69 runners, 4 hosts), diagnose from persisted step/task state, fix with a regression test, redeploy, rerun. Warm caches made each rerun cost roughly one hour of wall clock; image rollout added ~25 minutes.

Round 1 — the workflow failed; the catch never engaged. Two bugs:

- **B1: the state-changer loop advances before executing.** The transition into `CATCH_BEGIN` was written with a pending state-change request; the changer loop's contract is "advance *then* execute," so the handler that creates the recovery sub-block was skipped entirely and execution landed in `CATCH_CONTINUE` with nothing to wait for.
- **B2: recovery counted the corpse as its own failure.** `CatchContinue` enumerated *all* child blocks of the step as catch sub-blocks — including the errored `andThen` body whose failure had triggered the catch. Every declaration-level catch therefore concluded "recovery failed" even when its own steps succeeded.

Round 2 — the catch engaged and created its sub-block; the recovery step itself errored: `Attribute 'error' not found on step 'one'`.

- **B3: commit-visibility race on the error pseudo-returns.** The caught error is stored as synthetic return attributes on the step, and recovery code reads them via `$.error`. The write joined the iteration's *batched* commit; the recovery sub-block — processed by another runner, or later in the same iteration — resolved against the *persisted* step, which did not yet have them.

Round 3 — with eager persistence, the failure moved: the same "attribute not found" error, but now the attribute demonstrably existed on the persisted step. And separately, the *original facet body* re-executed inside the catch block on a second runner.

- **B4a: name-based container resolution under fan-out.** `$.attr` on a container resolved returns *by the container's statement name*. Under a 25-iteration `foreach`, twenty-five steps share the name `one`; the resolver found a completed *sibling iteration's* step — which had normal returns but no error pseudo-returns. Resolution now carries the container's step identity, not its name.
- **B4b: recovery code existed only in one process's memory.** The `AND_CATCH` sub-block's body AST was cached in-memory by the runner that created it. When another runner claimed the sub-block's continuation, the cold-cache fallback re-derived the container's *default* body — and re-executed the entire failed region inside the recovery block. Catch bodies are now re-derivable from the persisted program, as `foreach` and conditional sub-blocks already were.

Round 4 — recovery executed correctly and produced its results — into the void: the step had already completed, empty.

- **B5a: a race between recovery creation and its observer.** A step's `CATCH_CONTINUE` could be processed (by another runner, or a liveness nudge) before the iteration that created the sub-block committed. Seeing zero sub-blocks, it advanced to capture with no yield. Fix: when a catch clause exists but its sub-block is not yet visible, wait — creation is idempotent by deterministic identity.
- **B5b: the database could never return the sub-block anyway.** `get_blocks_by_step` — the query every Continue handler uses to observe its children — carried a hardcoded `object_type` filter predating the catch feature: `{AndThen, AndMap, AndMatch, Block}`. `AndCatch` blocks were *unqueryable on MongoDB*, period. With B5a's wait in place, this parked steps forever; before it, it was the silent enabler of B2's miscount.

After the fifth fix the workflow completed with the designed outcome — Greenland disclosed as excluded, all other regions ranked. The catch recovery required no further fixes in any subsequent run, including two later full reruns of the same workflow under unrelated changes. We are careful not to overclaim here: those reruns still required operator babysitting at the fan-in tail — but for a *separate, unrelated* defect (a continuation-chain liveness stall, at the time of writing an open problem) that has nothing to do with catch recovery. What the campaign fixed stayed fixed; what it did not touch remained broken, which is the correct outcome to report.

Cost accounting. Seven defects, four reruns: roughly six hours of wall-clock verification (mostly unattended), five fixes across four image rollouts, and diagnosis performed entirely from persisted step/task state — no debugger, no log archaeology beyond `docker logs`. Per-defect find-and-fix cost was well under a working day. Set against a shipped, documented, fully-tested feature that did not work at all, the economics are not close.

4. The taxonomy: why the tests were blind

Five of the seven defects map to a semantic property in which a single-address-space fake *cannot* faithfully represent a coordination substrate (G1–G5); the remaining two are a control-flow path-coverage class (G6) that we treat separately. We state each gap generally, then the mapping.

G1 — Object aliasing vs serialization. A fake returns live references; mutations are visible to every reader instantly, before any "save." A real store serializes: a write is invisible until committed, and every read is a private copy. *Blinded:* B3 (the eager-persist race cannot exist when mutation is publication). Contributed to B5a.

G2 — Index semantics. A fake's secondary indexes are usually structural (`is_block` flags, dictionary buckets) while the real store's are *queries with predicates* that must be maintained as the schema grows. The fake answers "what are this step's blocks?" by construction; the database answers whatever its filter says. *Blinded:* B5b — MemoryStore indexed by `is_block` and returned `AndCatch` from the day the feature was written; MongoDB's hardcoded type list never did. Every in-process test of catch observation was testing the fake's generosity.

G3 — Commit visibility and ordering. In one address space there is no window between "decided" and "durable." Distributed, every batched commit opens a window in which another process acts on the old world. *Blinded:* B3, B5a. Both are textbook read-your-writes violations that are *definitionally* unobservable in-process.

G4 — Name vs identity resolution. In a single test workflow, names are unique and resolution by name is resolution by identity. Under production fan-out, names are *roles* shared by many step instances, and any name-keyed lookup is a latent wrong-instance bug. *Blinded:* B4a. No in-process test ran a catch under a wide enough foreach with completed siblings — and even one that did might pass, since the fake returns whichever instance the dictionary yields.

G5 — Cache locality. In-process, every cache is shared by all "workers" because there is only one process. Distributed, a cache entry exists on the process that created it, and correctness requires every derived artifact to be re-derivable from persistent state. *Blinded:* B4b. The AST cache was correct-by-construction in every test.

G6 — Control-loop interleaving (the near-miss class). B1 and B2 were not store-semantics bugs — they were reachable in principle in-process. They were blinded instead by *path coverage*: the unit tests drove catch through the leaf-failure entry point (which handled the transition correctly), while distributed execution reached it through block-error propagation (which did not). The state machine had two doors; the tests knocked on one. We include this class because it co-travels with the others: distributed execution does not merely add new failure semantics, it *selects different code paths* through machinery the tests believed covered.

Two observations about the taxonomy. First, G1–G5 are not defects of effort in the fake: a fake that serialized every object, simulated commit windows, and enforced re-derivability would be approaching a database simulator, at which point its value as a fast test double is gone. The gaps are the *price* of the speed. Second, the gaps compound: B5b (G2) hid behind B2 (G6), which hid behind B1 (G6); each fix was necessary to *reach* the next bug. This is why the campaign took four rounds rather than one — and why static inspection of any single layer would not have found the set.

5. Mitigations and their measured effect

M1 — Same-assertion dual-store parity suites. For every store method a Continue handler or claim path depends on, one parametrized test runs *identical assertions* against `MemoryStore` and `MongoStore` (via mongomock). The G2 class becomes a test failure: the `get_blocks_by_step` regression pins that every block type round-trips through both stores, and the suite for the subsequent *environments* feature (claim filtering, script-task claiming, demand scans) was written parity-first: ~18 test functions, the store-facing ones parametrized across both backends. During that feature's development the parity suite caught a filter-shape divergence between the stores before any deployment; the feature's live pilot then passed on the first attempt for the claim-routing layer. The discipline is cheap: the parametrized fixture is ~10 lines, and the suites run in under a second.

M2 — Verification runs as first-class methodology. The campaign's retrospective rule, now recorded in the project's engineering doctrine: *a feature is not done until it has run distributed*. Concretely: every runtime feature ships with a designed live verification (a workflow whose success is only reachable through the new code path — e.g., an environment pinned to a library present in no image, so completion *proves* on-demand materialization). Three such designed verifications were run for the environments feature (placement-only pilot, bake-provisioned cold run, never-baked lazy run); each either passed or localized its bug within one round. The atlas rerun style — cached inputs, ~1h, unattended, diagnosis from persisted state — makes this affordable as routine practice rather than release ceremony.

What M1/M2 do *not* cover: G3 races remain fundamentally probabilistic; parity suites verify agreement of store semantics, not interleavings. The campaign found its G3 bugs because live fan-out at fleet scale samples interleavings aggressively — 25 concurrent iterations across 69 claimants is a better race generator than any unit harness we could write.

6. Related work and discussion

Test doubles. The fake-vs-mock literature (Fowler's taxonomy) treats fakes as the high-fidelity end of the double spectrum. Our finding sharpens this: for coordination substrates, even a *complete, behaviorally faithful* fake diverges on exactly the semantics coordination depends on. Fidelity of interface is not fidelity of physics.

Jepsen and fault injection. Jepsen-style testing attacks the database's own guarantees under partition. Our bugs sit a layer up — the *application's use* of a correctly-functioning database — and require no fault injection at all: every bug reproduced on a healthy fleet under ordinary concurrency.

Deterministic simulation testing (FoundationDB; TigerBeetle's VOPR; Antithesis) is the strongest known answer to G3-class bugs: run the real code against a simulated substrate with controlled, exhaustively explored interleavings. It requires architecting for simulation from the start (single-threaded state machines, injected time and I/O). Facetwork's runners were not so architected; the parity-suite + live-verification combination is the retrofit-friendly approximation. We note, however, that DST would *not* have caught B5b (G2) unless the simulated store shared the production store's query implementation — the gap taxonomy cuts across even that methodology.

Model checking (TLA+) verifies protocol designs; every bug here was an implementation infidelity *below* any reasonable protocol model. A TLA+ spec of catch recovery would itself have used identity-based resolution and a complete block query — the spec would have been correct and the system still broken.

Generalization. The preconditions for this failure pattern are common, not exotic: (a) an interface-abstracted persistence layer, (b) a fast in-memory implementation for tests, (c) correctness that lives in cross-process observation of persisted state. Any Temporal-style engine, job queue, saga coordinator, or step-function clone with a fake meets them. Our prescription, in one sentence each: audit every store method against the G1–G5 taxonomy; run identical assertions against both stores; design one live verification per coordination feature whose success is unreachable without the new path; and treat "all tests green" for coordination code as a statement about the fake, not the system.

7. Threats to validity

Single system, single campaign, one team's testing culture. The fake was written by the same project that wrote the production store, so gap correlation with *this* codebase's habits is possible — though G1/G3/G5 are properties of address spaces, not habits. Defect count (seven, five of them store-semantics) is small; we claim taxonomy coverage, not frequency statistics. The cost figures benefit from unusually cheap reruns (cached inputs); systems without replayable workloads will pay more per round, which strengthens rather than weakens the case for building replayability.

8. Conclusion

The catch clause worked perfectly in a world with one address space, one step named `one`, no commit windows, shared caches, and a generous index — a world that exists only inside the test suite. Roughly six hours of live verification bought what a green test suite could not: a feature that works where it runs. The taxonomy in §4 is offered as the audit checklist we wish we had had; the mitigations in §5, as evidence that the fix is procedural, cheap, and durable.

Artifacts: fix commits `6750740`, `d9225fa`, `37b3083`, `21208c0`, `9409256`; parity suites `tests/runtime/test_environment_routing.py`, `tests/runtime/test_mongo_store.py`; engineering doctrine `docs/architecture/lessons-learned.md` §23; verification workflow `osm.emergency.flows.ContinentalEmergencyAtlas` and its persisted run records.